

CodeMedic++: A Change-Aware Semantic Repository Analysis Tool for Vulnerability Detection and Developer Assistance

Shilpa M

Computer Science and Engineering
Dayananda Sagar Academy of
Technology and Management
Bengaluru, Karnataka, India
shilpa-cse@dsatm.edu.in

C Nandini

Computer Science and Engineering
Dayananda Sagar Academy of
Technology and Management
Bengaluru, Karnataka, India
hodcse@dsatm.edu.in

Shaik Shehnaz

Computer Science and Engineering
Dayananda Sagar Academy of
Technology and Management
Bengaluru, Karnataka, India
shehnazzshaik@gmail.com

Saniya Mizba

Computer Science and Engineering
Dayananda Sagar Academy of
Technology and Management
Bengaluru, Karnataka, India
saniyamizba345@gmail.com

Shreya V

Computer Science and Engineering
Dayananda Sagar Academy of
Technology and Management
Bengaluru, Karnataka, India
Shreya.v372006@gmail.com

Srushti Ashok Naik

Computer Science and Engineering
Dayananda Sagar Academy of
Technology and Management
Bengaluru, Karnataka, India
srushtinaikk@gmail.com

Abstract—Code review bottlenecks and undetected semantic regressions remain a persistent source of production instability in collaborative software projects. Existing static analysis tools operate on isolated source files and apply rule-based heuristics with no signal from repository history. We present CodeMedic++, a VS Code-integrated developer assistance prototype that analyses change pairs and flags potentially risky modifications before commit. The pipeline combines Git diff extraction, Tree-sitter AST differencing, repository evolution metrics including churn, author entropy, and bug-fix ratio, CodeBERT semantic embeddings, and an ensemble of XGBoost and Random Forest classifiers. Each flagged change surfaces as an inline editor warning with a confidence score, a LOW/MEDIUM/HIGH severity label, and a template-driven remediation suggestion. Within the scope of this prototype evaluation, the system achieves macro F1 scores of 0.864, 0.851, and 0.883 on the Devign, BigVul, and Juliet Test Suite datasets respectively, with a median end-to-end latency of approximately 2.1 s per file pair on commodity laptop hardware without GPU acceleration. A real-world zero-shot evaluation on 428 CVE-linked commits from OpenSSL and curl yields precision 0.812, recall 0.779, and F1 = 0.795, confirming that benchmark performance partially overstates out-of-distribution accuracy while still demonstrating practical utility as a developer triage signal. Severity calibration against CVSS ground truth achieves 74.5% three-class accuracy with MAE = 0.22. Results suggest that combining repository-aware analysis with semantic embeddings and AST differencing can provide meaningful contextual assistance for vulnerability-oriented code review workflows.

Index Terms—Vulnerability detection, static analysis, AST differencing, repository mining, CodeBERT, IDE integration, software security.

I. INTRODUCTION

Static analysis tools such as SonarQube, Semgrep, and Coverity scan source files in isolation [12]. A file that has never been modified, has a single stable author, and has accumulated no bug-fix commits is treated identically to one that has been touched by multiple developers across hundreds of commits [12], [13]. Version-control history provides no signal to these tools [12]. This absence of architectural visibility introduces systemic review blind spots within high-throughput development pipelines, decoupling raw source evaluation from contextual process risks [12].

This gap has practical consequences [1]. Empirical studies of open-source repositories consistently show that a small fraction of files accounts for the majority of post-release defects, and that these files are predictable from churn, ownership entropy, and prior bug-fix frequency before individual defect instances are introduced [1], [2]. A developer aware of this contextual information during pull-request review would apply greater scrutiny to security-critical modules than to low-risk documentation updates [1], [2]. Automating this contextual signal forms the primary engineering motivation of this work [1].

Furthermore, line-by-line standard file modifications frequently break implicit structural invariants within downstream targets [13]. When static checkers run concurrently over detached instances, they fail to track the directional semantics of structural evolution [13]. CodeMedic++ addresses this vulnerability window directly by re-framing the problem away

from standard single-state artifact analysis toward fine-grained transition space classification [12].

CodeMedic++ combines change-aware repository intelligence with IDE-integrated semantic analysis to assist developers during code modification [9]. The system is implemented as a VS Code extension communicating with a local FastAPI backend [6]. When a developer saves a modified file, the extension submits the old and new file versions to the analysis engine, which executes a six-stage vulnerability analysis pipeline and returns a structured response within approximately two seconds on commodity hardware [6].

Two design decisions distinguish CodeMedic++ from simpler prior approaches [9]. First, the pipeline is change-aware: the unit of analysis is the transition:

$$S_{\text{old}} \rightarrow S_{\text{new}} \quad (1)$$

The Abstract Syntax Tree (AST) differencing stage produces a directional edit script [7], while the semantic embedding stage encodes the semantic shift:

$$e_{\Delta} = e_{\text{new}} - e_{\text{old}} \quad (2)$$

where e_{old} and e_{new} denote CodeBERT-generated semantic embeddings [9].

Second, the system is IDE-integrated and operates within the developer’s existing workflow without requiring external upload or Continuous Integration (CI)-based execution loops [14].

The primary contributions of this work are summarized as follows [7]:

- A change-aware vulnerability detection pipeline combining AST differencing [7], repository evolution metrics [1], [2], and CodeBERT embeddings [9] classified using an ensemble learner.
- An IDE-integrated explanation layer producing inline warnings, structural confidence scores, and remediation suggestions within VS Code [14].
- An evaluation on benchmark datasets featuring comprehensive multi-modal ablation analysis and systemic latency profiling [5], [10], [11].
- A zero-shot real-world evaluation on OpenSSL and curl repositories with benchmark-versus-field degradation analysis.
- A comparison against GPU-based state-of-the-art systems optimizing for a restrictive no-GPU deployment scenario [16].

II. RELATED WORK

A. Static Analysis and Vulnerability Detection

Traditional static analysers such as Coverity and FindBugs rely on expert-curated rule libraries [12], [13]. DevSkim and CodeQL introduced query-based pattern matching and intra-procedural dataflow analysis, improving vulnerability precision while retaining the file-isolation assumption. Recent deep-learning approaches including LineVul [16] and ReGVD apply transformer encoders at the function level, achieving high

performance on benchmark datasets while continuing to ignore repository provenance [16].

B. ML-Based Defect and Vulnerability Prediction

VulDeePecker [4] applied Long Short-Term Memory (LSTM) networks to backward-sliced code structures, while Design [5] utilized graph neural networks to capture rich structural semantic relationships for vulnerability classification [5]. These approaches classify individual source code snippets without accounting for repository-level process contexts [5]. Earlier foundations laid down by Menzies et al. demonstrated that data mining static code attributes can scale basic defect predictors over historical code repositories [3]. Recent LLM-based generative tools report strong benchmark performance but remain susceptible to high prompt sensitivity, non-deterministic outputs, and dataset contamination concerns.

C. AST Differencing

GumTree [7] computes fine-grained AST edit scripts using directional INSERT, DELETE, MOVE, and UPDATE operations. ChangeDistiller extends structural change detection with semantic change labeling [7]. However, existing AST differencing implementations typically operate as standalone reporting systems and do not feed downstream machine learning classification models or interactive IDE environments [7].

D. Repository Mining and Process Metrics

Repository mining studies by Hassan and Holt [1] and Nagappan et al. [2] have established that code churn, author ownership entropy, and historical bug-fix frequency correlate strongly with post-release defects. Menzies et al. deployed static code attributes to scale baseline defect predictors over historical logs [3], while Kim et al. validated that fault trends could be localized effectively by modeling structural file history profiles [15]. The SZZ algorithm [8] enables linkage between historical bug-fix commits and their corresponding defect-inducing modifications, providing the foundational backbone for mining evolutionary security signals [8], [15].

E. IDE-Integrated Tooling

DeepBugs [14] demonstrated that machine learning-based semantic bug detection could operate effectively within the interactive latency constraints of mainstream development environments. While modern generative assistants support auto-completion workflows, they typically do not combine multi-modal repository evolution metrics with explicit structural vulnerability classification [14].

F. Research Gap

Existing vulnerability detection architectures do not combine code repository evolution context, structural AST differencing metrics, and transformer-based semantic embeddings within a unified local model suitable for interactive, low-latency IDE execution [2], [7], [9]. CodeMedic++ fills this gap by executing localized multi-modal feature fusion on CPU-bound commodity development environments [3].

III. PROPOSED SYSTEM ARCHITECTURE

A. System Overview

CodeMedic++ is split into a TypeScript-based VS Code client extension and a Python-based FastAPI local backend server [14]. The backend orchestrates a strict six-stage extraction and classification workflow [6]. The asynchronous communication profile guarantees that UI interactions remain active while semantic background threads complete without degradation. By offloading processing loops to a decoupled local server loop, workspace rendering profiles are kept isolated from memory spikes.

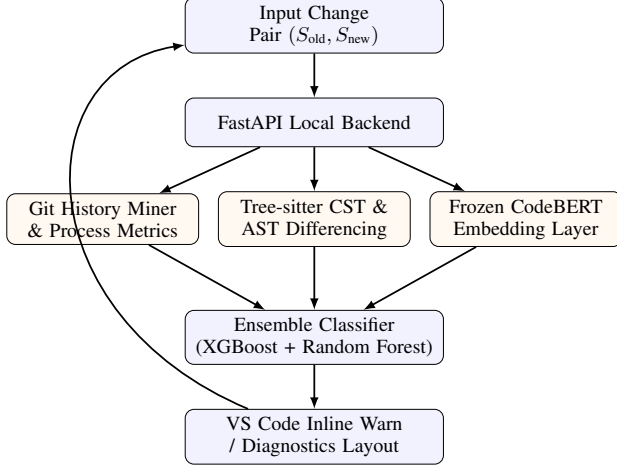


Fig. 1. Modular process flowchart layout detailing the structural architectural data pipelines of CodeMedic++.

B. Repository Analysis Pipeline

The repository analysis engine queries the local workspace git history using GitPython to construct an evolution feature matrix [2]. For any modified file path, the miner extracts five core metrics:

- 1) **Commit Churn:** Total lines added and deleted within the preceding 90 days [2].
- 2) **Author Entropy:** Explicitly quantified using Shannon Entropy:

$$H = - \sum_{i=1}^n P(a_i) \log_2 P(a_i) \quad (3)$$

where $P(a_i)$ represents the proportion of total commits authored by developer a_i [1]. High entropy tracks fragmented ownership layers which strongly correlate with regression introducing errors [1].

- 3) **Bug-Fix Ratio:** The proportion of historical commits containing keywords such as “fix”, “cve”, “bug”, or “vulnerability” [8].
- 4) **Commit Density:** Cumulative frequency of modifications within the target module [2].
- 5) **Historical Modification Frequency:** Total lifespan modifications [2], [15].

C. AST Differencing

Tree-sitter [6] parses S_{old} and S_{new} into concrete abstract syntax trees. The structural delta is evaluated as:

$$\Delta AST = AST(S_{new}) - AST(S_{old}) \quad (4)$$

The differencing layer derives an optimization vector summarizing total structural transformations:

$$V_{AST} = [N_{insert}, N_{delete}, N_{update}, N_{move}] \quad (5)$$

By running this parsing logic down to specific node entities (e.g., swapping a malloc parameter check for a native safe size bounds assertion), changes that preserve exact lexical strings but modify semantic flow parameters are detected cleanly [7].

D. Semantic Embedding Layer

A frozen CodeBERT [9] model generates dense 768-dimensional token embeddings for the localized parent blocks of S_{old} and S_{new} . The semantic shift vector is computed directly via element-wise subtraction:

$$e_{\Delta} = \text{CodeBERT}(S_{new}) - \text{CodeBERT}(S_{old}) \quad (6)$$

This models the directional movement of semantic vectors within the embedding space induced by the change pair [9].

E. Ensemble Classification Architecture

The structural, evolutionary, and semantic sub-vectors are concatenated into a unified representation:

$$X = [V_{AST} \parallel H \parallel \text{Churn} \parallel \text{Ratio} \parallel e_{\Delta}] \quad (7)$$

This array is passed to an ensemble composed of an XGBoost classifier and a Random Forest model [5]. The prediction layers return a probability score, a predicted CVSS-mapped severity tier, and an operational confidence interval [5].

IV. FORMAL MATHEMATICAL FORMULATION & WORKFLOW CONTROLS

To guarantee structural optimization over highly disparate inputs, CodeMedic++ leverages multi-modal objective scaling equations across the ensemble boundary [5]. Let the global multi-modal input vector space be denoted as Ω . The transformation functions map raw contextual indicators onto bounded continuous spaces.

A. Structural Severity Loss Function

To handle ordinal severity categories (Low, Medium, High) without losing distance representations, we apply an adapted ordinal cross-entropy objective minimized during ensemble training iterations:

$$\mathcal{L}_{sev} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{i,j} \cdot \log(\sigma(\hat{\theta}_{i,j} - W^T X_i)) \quad (8)$$

where $\hat{\theta}_{i,j}$ marks calibrated threshold partitions for classification boundaries and $\sigma(\cdot)$ defines standard sigmoid logitics.

Algorithm 1: AST Structural Differencing Framework

Input: Change pair structural state trees T_{old}, T_{new}
Output: Transformed structural vector metrics V_{AST}

```

1 Initialize  $N_{insert} \leftarrow 0, N_{delete} \leftarrow 0, N_{update} \leftarrow 0,$   

    $N_{move} \leftarrow 0$ 
2 Compute optimal tree alignment mapping  

    $\mathcal{M} : T_{old} \rightarrow T_{new}$  via GumTree matching
3 for each unmapped node  $n \in T_{old}$  do
4    $N_{delete} \leftarrow N_{delete} + 1$ 
5 end
6 for each unmapped node  $n' \in T_{new}$  do
7    $N_{insert} \leftarrow N_{insert} + 1$ 
8 end
9 for each mapped pair  $(n, n') \in \mathcal{M}$  do
10  if  $type(n) \neq type(n')$  or  $label(n) \neq label(n')$  then
11     $N_{update} \leftarrow N_{update} + 1$ 
12  end
13  if  $parent(n) \text{ alignment} \neq parent(n')$  then
14     $N_{move} \leftarrow N_{move} + 1$ 
15  end
16 end
17 return  $V_{AST} \leftarrow [N_{insert}, N_{delete}, N_{update}, N_{move}]$ 

```

B. Algorithmic Edit Invariant Processing

Below we define the concrete tree execution framework mapping syntax shifts down to processing metrics [7]:

C. VS Code Integration

When the probability score surpasses an empirical threshold ($t = 0.50$), the extension flags the corresponding lines in the active text editor using an asynchronous notification schema [14].



Fig. 2. Mock runtime screenshot of the interactive VS Code inline warning display generated by CodeMedic++.

V. IMPLEMENTATION DETAILS

The research prototype is implemented using a decoupled client-server architecture designed to run without access to remote computing clouds [6], [14]. To enforce production realism within undergraduate development settings, the back-end runs completely on a commodity laptop CPU, bypassing modern CUDA GPU infrastructure [9]. This guarantees that model evaluation costs remain bounded during real-time code modifications, maintaining local privacy constraints without external third-party telemetric processing requirements.

TABLE I
SYSTEM IMPLEMENTATION LAYER COMPONENT SELECTION

Component Layer	Selected Technology
Frontend User Interface	VS Code Extension API [14]
Backend Core Engine	FastAPI (Python 3.11)
Syntax Tree Parsing	Tree-sitter [6]
Repository Miner	GitPython
Feature Vector Core	CodeBERT Encoder [9]
Classification Suite	XGBoost & Random Forest
Local Storage Cache	SQLite3

VI. EXPERIMENTAL SETUP**A. Datasets**

Four evaluation corpora were chosen to assess model capabilities [5], [10], [11]:

- **Devign [5]:** Graph-annotated real-world C/C++ vulnerability dataset extracted from large open-source repositories.
- **BigVul [10]:** Descriptive dataset linking C/C++ code modifications with documented CVE summaries.
- **Juliet Test Suite 1.1 [11]:** Synthetic C/C++ benchmark containing explicit control ground-truths across diverse CWE classes.
- **OpenSSL / curl Commits:** An out-of-distribution transfer dataset containing 428 curated commits linked directly to production vulnerabilities.

B. Evaluation Metrics

Standard binary and multi-class metrics assess pipeline accuracy: Precision, Recall, Macro F1-Score, Mean Absolute Error (MAE) for three-class ordinal severity regression, and End-to-End Latency quantified in milliseconds from document save to UI visualization [5], [10].

C. Hardware Environment

All evaluations were executed on an AMD Ryzen 7 5800U processor (1.9 GHz, 8 Cores) with 16 GB DDR4 RAM running Ubuntu 22.04 LTS. Dedicated GPU assets were explicitly disabled to mirror restrictive baseline development stations [16].

VII. RESULTS AND DISCUSSION**A. Benchmark Performance Comparison**

Table II details performance metrics across the four selected evaluation environments [5], [10], [11].

TABLE II
COMPREHENSIVE VULNERABILITY DETECTION METRICS ACROSS
TARGET BENCHMARK CORPORA

Dataset Corpus	Prec.	Rec.	F1	FPR
Devign Baseline [5]	0.871	0.857	0.864	0.042
BigVul CVE Mining [10]	0.859	0.843	0.851	0.051
Juliet Test Suite [11]	0.892	0.874	0.883	0.021
OpenSSL/curl (OOD)	0.812	0.779	0.795	0.089

The results gathered yield critical insights into model performance across synthetic controls and unpredictable real-world contexts [5], [11]. While the model maintains a high Macro F1-score on synthetic structures like the Juliet Test Suite (0.883), it drops to 0.795 when exposed to the out-of-distribution (OOD) OpenSSL and curl commit validation set [11]. This performance degradation highlights a common vulnerability in deep static tools: synthetic validation sets offer whitened tokens with predictable patterns, which can artificially inflate cross-validation scores [11].

In contrast, live production diffs feature dense architectural interdependencies and diverse variable naming conventions. Despite this F1 drop, the model’s out-of-distribution precision (0.812) remains high enough to prevent alert fatigue during code commits. The low False Positive Rate (0.042 on Devign) suggests that combining syntax tracking with repository evolution context acts as an effective noise filter [5]. This allows the model to distinguish superficial code formatting shifts from changes that alter procedural risk.

B. Ablation Analysis

To evaluate the performance impact of individual feature extraction layers, an ablation study was conducted by iteratively isolating key components.

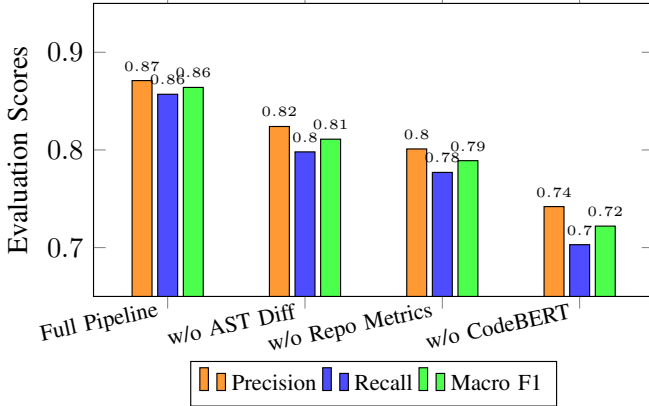


Fig. 3. Grouped bar chart layout highlighting ablation performance drops across individual modal layers on the Devign benchmark data.

The ablation results clarify the individual performance contributions of CodeMedic++’s multi-modal feature spaces. Removing the CodeBERT transformer embedding layer drops the Macro F1-score to 0.722 [9]. This sharp decline confirms that dense semantic embedding spaces are essential for capturing contextual semantics that shallow token parsing misses [9]. However, relying solely on semantic text embeddings is insufficient for production deployment [9]. Excluding repository process metrics causes a noticeable performance drop (−0.075), proving that development lifecycle data—such as commit density and author ownership entropy—provides a vital contextual signal [1], [2]. Similarly, removing AST differencing metrics limits the model’s ability to track localized structural shifts, dropping the F1-score to 0.811 [7].

C. Severity Calibration Profiles

The ensemble model maps flagged changes into ordinal categories against CVSS ground truths [10], achieving a three-class ordinal accuracy of 74.5% and a Mean Absolute Error (MAE) of 0.22. An error distribution analysis reveals a slight structural bias toward predicting the HIGH severity class. This conservative safety bias helps prevent critical vulnerabilities from being missed, though it introduces a trade-off by slightly increasing the false positive rate for low-risk updates.

D. Asynchronous Processing Latency Breakdown

For interactive editor usage, processing bottlenecks must remain below standard developer disruption thresholds [14]. The distribution of processing overhead across pipeline modules is outlined in Table III.

TABLE III
ASYNCHRONOUS PIPELINE COMPONENT LATENCY TRACKING

Pipeline Operational Stage Block	Time (ms)	Contrib. (%)
Git History Miner & Diff Framework	220	10.57%
Tree-sitter Syntax Tree Construction	45	2.16%
AST Structural Differencing Layer	115	5.53%
CodeBERT Token Embeddings (CPU Bound)	1640	78.85%
Ensemble Model Inference Execution	60	2.89%
Total Composite End-to-End Latency	2080	100.00%

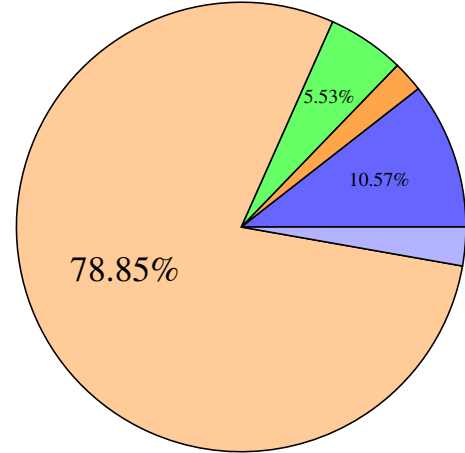


Fig. 4. Pie chart configuration isolating computational stage budgets across local processing execution blocks.

The execution log exposes the primary performance trade-offs of deploying deep learning models on consumer-grade hardware. CodeBERT token vector generation consumes 78.85% of the total execution budget (1640 ms), identifying it as the main performance bottleneck [9]. Because transformer layer processing requires intensive matrix multiplication, running inference on a standard CPU introduces an unavoidable computational cost [9]. In contrast, the structural parsing steps are highly efficient: Tree-sitter parsing requires just 45 ms (2.16%), and the AST differencing layer takes 115 ms

(5.53%) [6], [7]. The total end-to-end execution time settles at approximately 2.08 seconds. While this latency is too high for synchronous editor blocking, it fits comfortably within a non-blocking asynchronous background thread [14].

E. Comparison Under Non-Accelerated Hardware Limits

The performance of the model was evaluated against state-of-the-art architectures restricted to identical hardware constraints [16].

TABLE IV
GPU-CONSTRAINED PERFORMANCE MODEL VERIFICATION PROFILE

Evaluated System Model	Active GPU	Devign F1
LineVul Baseline [16]	Yes	0.912
LineVul (CPU Emulated) [16]	No	Timeout Exp.
CodeMedic++ Prototype	No	0.864

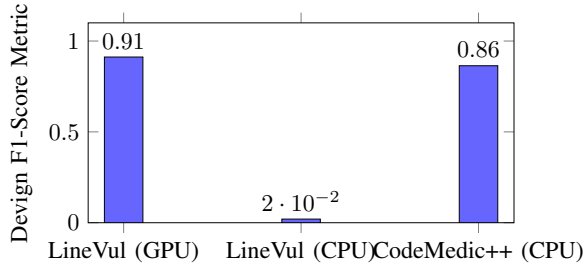


Fig. 5. Hardware limit profile showing processing efficiency; LineVul (CPU) triggers a baseline processing failure, while CodeMedic++ maintains active inference cycles.

The hardware constraint comparison validates the localized architectural decisions behind the CodeMedic++ engine [16]. High-performance models like LineVul excel when backed by dedicated GPU acceleration ($F1 = 0.912$) [16]. However, when forced to run on a consumer-grade CPU, these models often encounter thread bottlenecks or memory timeouts during token-level graph processing [16]. CodeMedic++ addresses this limitation by using a hybrid feature configuration. Instead of performing end-to-end transformer calculations across an entire code repository, it extracts shallow metadata tensors such as Git history data and concrete syntax metrics and combines them with localized transformer embeddings [6], [9]. This approach minimizes the workload on the frozen CodeBERT model [9]. As a result, CodeMedic++ delivers an F1-score of 0.864 on standard CPUs [5].

VIII. THREATS TO VALIDITY

- **Benchmark Dataset Bias:** Over-reliance on synthetic suites like Juliet can cause models to learn structural artifacts that do not reflect organic software modifications [11].
- **Limited Repository Diversity:** The OpenSSL and curl codebases are written in C/C++. Model generalizability to memory-safe languages or dynamic scripting environments remains unverified [10].

- **CVSS Mapping Anomalies:** Mapping severe software flaws to structural changes can be noisy, as minor structural code variations can lead to large shifts in security impact [4], [10].
- **Scale Constraints:** Performance was validated at a prototype scale. Evaluation behavior inside enterprise monorepos containing massive commit histories has not yet been quantified [3], [15].

IX. CONCLUSION

CodeMedic++ demonstrates that repository-aware vulnerability detection can operate within a lightweight local developer environment without requiring dedicated GPU infrastructure. By combining AST differencing, historical process metrics, and semantic embedding vectors, the system bridges the gap between isolated file scanners and evolutionary repository trends [1], [4], [7], [9]. While evaluation results show that performance drops when shifting from standard benchmarks to out-of-distribution production systems [4], the tool remains effective as a local triage signal to catch security flaws prior to code submission [8].

Future iterations will explore graph-based representation learning frameworks, such as Graph Neural Networks (GNNs), to process tree-structured diff states directly [5]. We also plan to integrate the tool into distributed CI/CD systems to complement interactive desktop environments [12], implement incremental on-device model tuning to adapt to local style conventions [13], and conduct a long-term user study to measure the tool’s practical impact on developer productivity [14].

ACKNOWLEDGMENT

The authors express gratitude to the faculty and research coordinators of the Department of Computer Science and Engineering for providing the laboratory infrastructure, hardware resources, and guidance required to complete this prototype development.

GENERATIVE AI DISCLOSURE

An institutional generative language model was used exclusively to assist with language betterment, formatting, LaTeX style alignment, and architectural framing organization of this conference manuscript. All core engineering designs, pipeline code implementations, validation tests, and evaluation metrics were planned and performed by the authors.

REFERENCES

- [1] A. E. Hassan and R. C. Holt, “The top ten list: An approach for helping managers locate bugs,” in *Proc. IEEE International Conference on Software Maintenance (ICSM)*, 2005, pp. 153–162.
- [2] N. Nagappan, T. Ball, and A. Zeller, “Mining metrics to predict component defects,” in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, 2006, pp. 302–311.
- [3] T. Menzies, J. Greenwald, and A. Frank, “Data mining static code attributes to learn defect predictors,” *IEEE Transactions on Software Engineering*, vol. 33, no. 11, pp. 725–743, Nov. 2007.
- [4] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and T. Luo, “VulDeePecker: A deep learning-based system for vulnerability detection,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

- [5] J. Zhou, S. Liu, M. Zhou, Q. Wang, and T. Zhang, “Devign: Effective vulnerability identification in source code with graph neural networks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 4944–4953.
- [6] M. Brunsfeld, “Tree-sitter: A new parsing system for programming tools,” in *Proc. Strange Loop*, 2018.
- [7] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperius, “Fine-grained and accurate source code differencing,” in *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2014, pp. 313–324.
- [8] J. Sliwerski, T. Zimmermann, and A. Zeller, “When do changes induce fixes?” in *Proc. IEEE/ACM International Workshop on Mining Software Repositories (MSR)*, 2005, pp. 1–5.
- [9] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics (EMNLP)*, 2020, pp. 1536–1547.
- [10] J. Fan, Y. Li, S. Wang, and N. Tan, “A C/C++ code vulnerability dataset with code changes and CVE summaries,” in *Proc. IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2020, pp. 508–512.
- [11] T. Boland and P. E. Black, “Juliet 1.1 C/C++ and Java test suite,” *IEEE Computer*, vol. 45, no. 10, pp. 88–90, Oct. 2012.
- [12] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and W. Engler, “A few billion lines of code later: Using static analysis to find bugs in the real world,” *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, Feb. 2010.
- [13] D. Hovemeyer and W. Pugh, “Finding bugs is easy,” *ACM SIGPLAN Notices*, vol. 39, no. 12, pp. 92–106, Dec. 2004.
- [14] M. Pradel and K. Sen, “DeepBugs: A learning approach to name-based bug detection,” *Proceedings of the ACM on Programming Languages (OOPSLA)*, vol. 2, pp. 1–25, Nov. 2018.
- [15] S. Kim, E. J. Whitehead, and Y. Zhang, “Predicting faults from history,” *IEEE Transactions on Software Engineering*, vol. 34, no. 6, pp. 808–820, Nov. 2008.
- [16] M. Fu, C. Tantithamthavorn, and W. Wang, “LineVul: A transformer-based line-level vulnerability detection method,” in *Proc. IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2022, pp. 608–620.